

MIT License

#

Copyright (C) IBM Corporation 2019

#

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated

documentation files (the "Software"), to deal in the Software without restriction, including without limitation the

rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit

persons to whom the Software is furnished to do so, subject to the following conditions:

#

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

Software.

#

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

SOFTWARE.

""""

Basic mechanisms for achieving differential privacy, the basic building blocks of the library.

""""

```
from diffprivlib.mechanisms.base import DPMachine, DPMechanism,
TruncationAndFoldingMixin

from diffprivlib.mechanisms.binary import Binary

from diffprivlib.mechanisms.bingham import Bingham

from diffprivlib.mechanisms.exponential import Exponential, ExponentialCategorical,
ExponentialHierarchical, \
PermuteAndFlip

from diffprivlib.mechanisms.gaussian import Gaussian, GaussianAnalytic,
GaussianDiscrete

from diffprivlib.mechanisms.geometric import Geometric, GeometricFolded,
GeometricTruncated

from diffprivlib.mechanisms.laplace import Laplace, LaplaceBoundedDomain,
LaplaceBoundedNoise, LaplaceFolded, \
LaplaceTruncated

from diffprivlib.mechanisms.snapping import Snapping

from diffprivlib.mechanisms.staircase import Staircase

from diffprivlib.mechanisms.uniform import Uniform

from diffprivlib.mechanisms.vector import Vector

# MIT License

#

# Copyright (C) IBM Corporation 2019

#

# Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated

# documentation files (the "Software"), to deal in the Software without restriction, including
without limitation the

# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
the Software, and to permit
```

persons to whom the Software is furnished to do so, subject to the following conditions:

#

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

Software.

#

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

SOFTWARE.

"""

Base classes for differential privacy mechanisms.

"""

import abc

from copy import copy

import inspect

from numbers import Real

from diffprivlib.utils import check_random_state

class DPMachine(abc.ABC):

"""

Parent class for :class:`.DPMechanism` and :class:`.DPTransformer`, providing and specifying basic functionality.

"""

```
@abc.abstractmethod
```

```
def randomise(self, value):
```

```
    """Randomise `value` with the mechanism.
```

```
    Parameters
```

```
    -----
```

```
    value : int or float or str or method
```

```
    The value to be randomised.
```

```
    Returns
```

```
    -----
```

```
    int or float or str or method
```

```
    The randomised value, same type as `value` .
```

```
    """
```

```
def copy(self):
```

```
    """Produces a copy of the class.
```

```
    Returns
```

```
    -----
```

```
    self : class
```

```
    Returns the copy.
```

```
    """
```

```
    return copy(self)
```

```
class DPMechanism(DPMachine, abc.ABC):
```

```
    r"""Abstract base class for all mechanisms. Instantiated from :class:`.DPMachine` .
```

```
    Parameters
```

```
    -----
```

```
    epsilon : float
```

```
    Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $[0, \infty]$ .
```

delta : float

Privacy parameter :math:`\delta` for the mechanism. Must be in [0, 1]. Cannot be simultaneously zero with

``epsilon``.

random_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random_state`` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, delta, random_state=None):
```

```
    self.epsilon, self.delta = self._check_epsilon_delta(epsilon, delta)
```

```
    self.random_state = random_state
```

```
    self._rng = check_random_state(random_state, True)
```

```
def __repr__(self):
```

```
    attrs = inspect.getfullargspec(self.__class__).kwonlyargs
```

```
    attr_output = []
```

```
    for attr in attrs:
```

```
        attr_output.append(attr + "=" + repr(self.__getattr__(attr)))
```

```
    return str(self.__module__) + "." + str(self.__class__.__name__) + "(" + ", ".join(attr_output) + ")"
```

```
@abc.abstractmethod
```

```
def randomise(self, value):
```

```
    """Randomise `value` with the mechanism.
```

Parameters

value : int or float or str or method

The value to be randomised.

Returns

int or float or str or method

The randomised value, same type as `value` .

"""

def bias(self, value):

"""Returns the bias of the mechanism at a given `value` .

Parameters

value : int or float

The value at which the bias of the mechanism is sought.

Returns

bias : float or None

The bias of the mechanism at `value` if defined, `None` otherwise.

"""

raise NotImplementedError

def variance(self, value):

"""Returns the variance of the mechanism at a given `value` .

Parameters

value : int or float

The value at which the variance of the mechanism is sought.

Returns

bias : float or None

The variance of the mechanism at `value` if defined, `None` otherwise.

"""

raise NotImplementedError

def mse(self, value):

"""Returns the mean squared error (MSE) of the mechanism at a given `value`.

Parameters

value : int or float

The value at which the MSE of the mechanism is sought.

Returns

bias : float or None

The MSE of the mechanism at `value` if defined, `None` otherwise.

"""

return self.variance(value) + (self.bias(value)) ** 2

@classmethod

def _check_epsilon_delta(cls, epsilon, delta):

if not isinstance(epsilon, Real) or not isinstance(delta, Real):

raise TypeError("Epsilon and delta must be numeric")

if epsilon < 0:

raise ValueError("Epsilon must be non-negative")

if not 0 <= delta <= 1:

raise ValueError("Delta must be in [0, 1]")

if epsilon + delta == 0:

raise ValueError("Epsilon and Delta cannot both be zero")

return float(epsilon), float(delta)

```

def _check_all(self, value):
    del value
    self._check_epsilon_delta(self.epsilon, self.delta)
    return True

class TruncationAndFoldingMixin: # pylint: disable=too-few-public-methods
    """Mixin for truncating or folding the outputs of a mechanism. Must be instantiated with a
    :class:`.DPMechanism`.

    Parameters
    -----

    lower : float
        The lower bound of the mechanism.

    upper : float
        The upper bound of the mechanism.
    """

    def __init__(self, *, lower, upper):
        if not isinstance(self, DPMechanism):
            raise TypeError("TruncationAndFoldingMachine must be implemented alongside a
            :class:`.DPMechanism`")

        self.lower, self.upper = self._check_bounds(lower, upper)

    @classmethod
    def _check_bounds(cls, lower, upper):
        """Performs a check on the bounds provided for the mechanism."""
        if not isinstance(lower, Real) or not isinstance(upper, Real):
            raise TypeError("Bounds must be numeric")

        if lower > upper:
            raise ValueError("Lower bound must not be greater than upper bound")

        return lower, upper

```



```

def _check_all(self, value):
    """Checks that all parameters of the mechanism have been initialised correctly"""
    del value

    self._check_bounds(self.lower, self.upper)

    return True

def _truncate(self, value):
    if value > self.upper:
        return self.upper
    if value < self.lower:
        return self.lower
    return value

def _fold(self, value):
    if value < self.lower:
        return self._fold(2 * self.lower - value)
    if value > self.upper:
        return self._fold(2 * self.upper - value)
    return value

def bernoulli_neg_exp(gamma, random_state=None):
    """Sample from Bernoulli( $\exp(-\text{gamma})$ ).
```

Adapted from "The Discrete Gaussian for Differential Privacy", Canonne, Kamath, Steinke, 2020.

<https://arxiv.org/pdf/2004.00010v2.pdf>

Parameters

gamma : float

Parameter to sample from Bernoulli($\exp(-\text{gamma})$). Must be non-negative.

random_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

Returns

One sample from the Bernoulli($\exp(-\text{gamma})$) distribution.

"""

```
if gamma < 0:
```

```
    raise ValueError("Gamma must be non-negative, got {gamma}.")
```

```
rng = check_random_state(random_state, True)
```

```
while gamma > 1:
```

```
    gamma -= 1
```

```
if not bernoulli_neg_exp(1, rng):
```

```
    return 0
```

```
    counter = 1
```

```
    while rng.random() <= gamma / counter:
```

```
        counter += 1
```

```
    return counter % 2
```

```
# MIT License
```

```
#
```

```
# Copyright (C) IBM Corporation 2019
```

```
#
```

```
# Permission is hereby granted, free of charge, to any person obtaining a copy of this  
# software and associated
```

```
# documentation files (the "Software"), to deal in the Software without restriction, including  
# without limitation the
```

rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit

persons to whom the Software is furnished to do so, subject to the following conditions:

#

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

Software.

#

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

SOFTWARE.

"""

The binary mechanism for differential privacy.

"""

import numpy as np

from diffprivlib.mechanisms.base import DPMechanism

from diffprivlib.utils import copy_docstring

class Binary(DPMechanism):

"""The classic binary mechanism in differential privacy.

Given a binary input value, the mechanism randomly decides to flip to the other binary value or not, in order to

satisfy differential privacy.

Paper link: <https://arxiv.org/pdf/1612.05568.pdf>

Parameters

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in $[0, \infty]$.

value0 : str

0th binary label.

value1 : str

1st binary label. Cannot be the same as ``value0``.

random_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

Notes

* The binary attributes, known as `labels`, must be specified as strings. If non-string labels are required (e.g.

integer-valued labels), a :class:`DPTTransformer` can be used (e.g. :class:`IntToString`).

"""

```
def __init__(self, *, epsilon, value0, value1, random_state=None):
```

```
    super().__init__(epsilon=epsilon, delta=0.0, random_state=random_state)
```

```
    self.value0, self.value1 = self._check_labels(value0, value1)
```

```
@classmethod
```

```
    def _check_labels(cls, value0, value1):
```

```
        if not isinstance(value0, str) or not isinstance(value1, str):
```

```
            raise TypeError("Binary labels must be strings. Use a DPTTransformer (e.g. transformers.IntToString) for "
```

```

"non-string labels")

if len(value0) * len(value1) == 0:

    raise ValueError("Binary labels must be non-empty strings")

if value0 == value1:

    raise ValueError("Binary labels must not match")

return value0, value1

def _check_all(self, value):

    super()._check_all(value)

    self._check_labels(self.value0, self.value1)

if not isinstance(value, str):

    raise TypeError("Value to be randomised must be a string")

if value not in [self.value0, self.value1]:

    raise ValueError(f"Value to be randomised is not in the domain {{\"{self.value0}\", \"{self.value1}\"}}, "
f"got \"{value}\"")

return True

@copy_docstring(DPMechanism.bias)

def bias(self, value):

    raise NotImplementedError

@copy_docstring(DPMechanism.variance)

def variance(self, value):

    raise NotImplementedError

def randomise(self, value):

    """Randomise `value` with the mechanism.

Parameters
-----

```

value : str

The value to be randomised.

Returns

str

The randomised value.

"""

```
self._check_all(value)
```

```
indicator = 0 if value == self.value0 else 1
```

```
unif_rv = self._rng.random() * (np.exp(self.epsilon) + 1)
```

```
if unif_rv > np.exp(self.epsilon) + self.delta:
```

```
    indicator = 1 - indicator
```

```
return self.value1 if indicator else self.value0
```

```
# MIT License
```

```
#
```

```
# Copyright (C) IBM Corporation 2019
```

```
#
```

```
# Permission is hereby granted, free of charge, to any person obtaining a copy of this  
# software and associated
```

```
# documentation files (the "Software"), to deal in the Software without restriction, including  
# without limitation the
```

```
# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of  
# the Software, and to permit
```

```
# persons to whom the Software is furnished to do so, subject to the following conditions:
```

```
#
```

```
# The above copyright notice and this permission notice shall be included in all copies or  
# substantial portions of the
```

Software.

#

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

SOFTWARE.

"""

The Bingham mechanism in differential privacy, for estimating the first eigenvector of a covariance matrix.

"""

import secrets

from numbers import Real

import numpy as np

from diffprivlib.mechanisms.base import DPMechanism

from diffprivlib.utils import copy_docstring

class Bingham(DPMechanism):

"""

The Bingham mechanism in differential privacy.

Used to estimate the first eigenvector (associated with the largest eigenvalue) of a covariance matrix.

Paper link: <http://eprints.whiterose.ac.uk/123206/7/simbingham8.pdf>

Parameters

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in $(0, \infty]$.

sensitivity : float, default: 1

The sensitivity of the mechanism. Must be in $[0, \infty]$.

random_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity=1.0, random_state=None):
```

```
    super().__init__(epsilon=epsilon, delta=0, random_state=random_state)
```

```
    self.sensitivity = self._check_sensitivity(sensitivity)
```

```
    if isinstance(self._rng, secrets.SystemRandom):
```

```
        self._rng = np.random.default_rng()
```

```
    @classmethod
```

```
    def _check_epsilon_delta(cls, epsilon, delta):
```

```
        if not delta == 0:
```

```
            raise ValueError("Delta must be zero")
```

```
        return super()._check_epsilon_delta(epsilon, delta)
```

```
    @classmethod
```

```
    def _check_sensitivity(cls, sensitivity):
```

```
        if not isinstance(sensitivity, Real):
```

```
            raise TypeError("Sensitivity must be numeric")
```

```
        if sensitivity < 0:
```

```
            raise ValueError("Sensitivity must be non-negative")
```

```
        return float(sensitivity)
```



```

def _check_all(self, value):
    super()._check_all(value)
    self._check_sensitivity(self.sensitivity)
    if not isinstance(value, np.ndarray):
        raise TypeError(f"Value to be randomised must be a numpy array, got {type(value)}")
    if value.ndim != 2:
        raise ValueError(f"Array must be 2-dimensional, got {value.ndim} dimensions")
    if value.shape[0] != value.shape[1]:
        raise ValueError(f"Array must be square, got {value.shape[0]} x {value.shape[1]}")
    if not np.allclose(value, value.T):
        raise ValueError("Array must be symmetric, supplied array is not.")
    return True

@copy_docstring(DPMechanism.bias)
def bias(self, value):
    raise NotImplementedError

@copy_docstring(DPMechanism.variance)
def variance(self, value):
    raise NotImplementedError

def randomise(self, value):
    """Randomise `value` with the mechanism.

    Parameters
    -----
    value : numpy array

    The data to be randomised.

    Returns
    -----

```

numpy array

The randomised eigenvector.

"""

self._check_all(value)

eigvals, eigvecs = np.linalg.eigh(value)

dims = value.shape[0]

if dims == 1:

return np.ones((1, 1))

if self.sensitivity / self.epsilon == 0:

return eigvecs[:, eigvals.argmax()]

value_translated = self.epsilon * (eigvals.max() * np.eye(dims) - value) / 4 / self.sensitivity

translated_eigvals = np.linalg.eigvalsh(value_translated)

left, right, mid = 1, dims, (1 + dims) / 2

old_interval_size = (right - left) * 2

while right - left < old_interval_size:

old_interval_size = right - left

mid = (right + left) / 2

f_mid = np.array([1 / (mid + 2 * eig) for eig in translated_eigvals]).sum()

if f_mid <= 1:

right = mid

if f_mid >= 1:

left = mid

b_const = mid

omega = np.eye(dims) + 2 * value_translated / b_const

omega_inv = np.linalg.inv(omega)

norm_const = np.exp(-(dims - b_const) / 2) * ((dims / b_const) ** (dims / 2))

```

while True:

    rnd_vec = self._rng.multivariate_normal(np.zeros(dims), omega_inv / 4,
    size=4).sum(axis=0)

    unit_vec = rnd_vec / np.linalg.norm(rnd_vec)

    prob = np.exp(-unit_vec.dot(value_translated).dot(unit_vec)) / norm_const\
    / ((unit_vec.dot(omega).dot(unit_vec)) ** (dims / 2))

    if self._rng.random() <= prob:

        return unit_vec

# MIT License

#

# Copyright (C) IBM Corporation 2019

#

# Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated

# documentation files (the "Software"), to deal in the Software without restriction, including
without limitation the

# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
the Software, and to permit

# persons to whom the Software is furnished to do so, subject to the following conditions:

#

# The above copyright notice and this permission notice shall be included in all copies or
substantial portions of the

# Software.

#

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE

```

```
# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER  
# LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
```

```
# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE  
# SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
```

```
# SOFTWARE.
```

```
"""
```

```
Implementation of the standard exponential mechanism, and its derivative, the  
hierarchical mechanism.
```

```
"""
```

```
from numbers import Real
```

```
import numpy as np
```

```
from diffprivlib.mechanisms.base import DPMechanism, bernoulli_neg_exp
```

```
from diffprivlib.mechanisms.binary import Binary
```

```
from diffprivlib.utils import copy_docstring
```

```
class Exponential(DPMechanism):
```

```
    """
```

```
    The exponential mechanism for achieving differential privacy on candidate selection, as  
    first proposed by McSherry
```

```
    and Talwar.
```

```
    The exponential mechanism achieves differential privacy by randomly choosing a  
    candidate subject to candidate
```

```
    utility scores, with greater probability given to higher-utility candidates.
```

Paper link: <https://www.cs.drexel.edu/~greenie/privacy/mdviadp.pdf>

Parameters

`epsilon` : float

Privacy parameter : ϵ for the mechanism. Must be in $(0, \infty]$.

`sensitivity` : float

The sensitivity in utility values to a change in a datapoint in the underlying dataset.

`utility` : list

A list of non-negative utility values for each candidate.

`monotonic` : bool, default: False

Specifies if the utility function is monotonic, i.e. that adding an individual to the underlying dataset can

only increase the values in `utility`.

`candidates` : list, optional

An optional list of candidate labels. If omitted, the zero-indexed list $[0, 1, \dots, n]$ is used.

`measure` : list, optional

An optional list of measures for each candidate. If omitted, a uniform measure is used.

`random_state` : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

```random_state``` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity, utility, monotonic=False, candidates=None,
measure=None,
```

```
 random_state=None):
```

```
 super().__init__(epsilon=epsilon, delta=0.0, random_state=random_state)
```

```
 self.sensitivity = self._check_sensitivity(sensitivity)
```

```
 self.utility, self.candidates, self.measure =
self._check_utility_candidates_measure(utility, candidates,
 measure)
```

```
 self.monotonic = bool(monotonic)
```

```
 self._probabilities = self._find_probabilities(self.epsilon, self.sensitivity, self.utility,
self.monotonic,
 self.measure)
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if not delta == 0:
```

```
 raise ValueError("Delta must be zero")
```

```
 return super()._check_epsilon_delta(epsilon, delta)
```

```
@classmethod
```

```
def _check_sensitivity(cls, sensitivity):
```

```
if not isinstance(sensitivity, Real):
```

```
 raise TypeError("Sensitivity must be numeric")
```

```
if sensitivity < 0:
```

```
 raise ValueError("Sensitivity must be non-negative")
```

```
return float(sensitivity)
```

```
@classmethod
```

```
def _check_utility_candidates_measure(cls, utility, candidates, measure):
```

```
 if not isinstance(utility, list):
```

```
 raise TypeError(f"Utility must be a list, got a {utility}.")
```

```
 if not all(isinstance(u, Real) for u in utility):
```

```
 raise TypeError("Utility must be a list of real-valued numbers.")
```

```
 if len(utility) < 1:
```

```
 raise ValueError("Utility must have at least one element.")
```

```
 if np.isinf(utility).any():
```

```
 raise ValueError("Utility must be a list of finite numbers.")
```

```
 if candidates is not None:
```

```
 if not isinstance(candidates, list):
```

```
 raise TypeError(f"Candidates must be a list, got a {type(candidates)}.")
```

```
 if len(candidates) != len(utility):
 raise ValueError("List of candidates must be the same length as the list of utility values.")
```

```
 if measure is not None:
```

```
 if not isinstance(measure, list):
```

```
 raise TypeError("Measure must be a list, got a {type(measure)}.")
```

```
 if not all(isinstance(m, Real) for m in measure):
```

```
 raise TypeError("Measure must be a list of real-valued numbers.")
```

```
 if np.isinf(measure).any():
```

```
 raise ValueError("Measure must be a list of finite numbers.")
```

```
 if len(measure) != len(utility):
```

```
 raise ValueError("List of measures must be the same length as the list of utility values.")
```

```
 return utility, candidates, measure
```

```
@classmethod
```

```
def _find_probabilities(cls, epsilon, sensitivity, utility, monotonic, measure):
```

```
 scale = epsilon / sensitivity / (2 - monotonic) if sensitivity / epsilon > 0 else float("inf")
```

```
 # Set max utility to 0 to avoid overflow on high utility; will be normalised out before
 returning
```

```
 utility = np.array(utility) - max(utility)
```



```

if np.isinf(scale):
 probabilities = np.isclose(utility, 0).astype(float)
else:
 probabilities = np.exp(scale * utility)

probabilities *= np.array(measure) if measure else 1
probabilities /= probabilities.sum()

return np.cumsum(probabilities)

def _check_all(self, value):
 super()._check_all(value)
 self._check_sensitivity(self.sensitivity)
 self._check_utility_candidates_measure(self.utility, self.candidates, self.measure)

 if value is not None:
 raise ValueError(f"Value to be randomised must be None. Got: {value}.")

 return True

@copy_docstring(DPMechanism.bias)
def bias(self, value):
 raise NotImplementedError

@copy_docstring(DPMechanism.variance)

```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
def randomise(self, value=None):
```

```
 """Select a candidate with differential privacy.
```

```
 Parameters
```

```

```

```
 value : None
```

```
 Ignored.
```

```
 Returns
```

```

```

```
 int or other
```

```
 The randomised candidate.
```

```
 """
```

```
 self._check_all(value)
```

```
 rand = self._rng.random()
```

```
 if np.any(rand <= self._probabilities):
```

```
 idx = np.argmax(rand <= self._probabilities)
```

```
 elif np.isclose(rand, self._probabilities[-1]):
```

```
 idx = len(self._probabilities) - 1
```

```
 else:
```

```

 raise RuntimeError("Can't find a candidate to return. "
 f"Debugging info: Rand: {rand}, Probabilities: {self._probabilities}")

 return self.candidates[idx] if self.candidates else idx

```

```
class PermuteAndFlip(Exponential):
```

```
 """
```

The permute and flip mechanism for achieving differential privacy on candidate selection, as first proposed by

McKenna and Sheldon.

The permute and flip mechanism is an alternative to the exponential mechanism, and achieves differential privacy by

randomly choosing a candidate subject to candidate utility scores, with greater probability given to higher-utility

candidates.

Paper link: <https://arxiv.org/pdf/2010.12603.pdf>

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

sensitivity : float

The sensitivity in utility values to a change in a datapoint in the underlying dataset.

utility : list

A list of non-negative utility values for each candidate.

monotonic : bool, default: False

Specifies if the utility function is monotonic, i.e. that adding an individual to the underlying dataset can

only increase the values in `utility`.

candidates : list, optional

An optional list of candidate labels. If omitted, the zero-indexed list [0, 1, ..., n] is used.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random\_state` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity, utility, monotonic=False, candidates=None,
random_state=None):
```

```
 super().__init__(epsilon=epsilon, sensitivity=sensitivity, utility=utility,
monotonic=monotonic,
```

```
 candidates=candidates, measure=None, random_state=random_state)
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```

@copy_docstring(DPMechanism.variance)
def variance(self, value):
 raise NotImplementedError

@classmethod
def _find_probabilities(cls, epsilon, sensitivity, utility, monotonic, measure):
 scale = epsilon / sensitivity / (2 - monotonic) if sensitivity / epsilon > 0 else float("inf")

 utility = np.array(utility)
 utility -= max(utility)

 if np.isinf(scale):
 log_probabilities = np.ones_like(utility) * (-float("inf"))
 log_probabilities[utility == 0] = 0
 else:
 log_probabilities = scale * utility

 return log_probabilities

def randomise(self, value=None):
 """Select a candidate with differential privacy.

```

Parameters

-----

value : None

Ignored.

Returns

-----

int or other

The randomised candidate.

"""

self.\_check\_all(value)

candidate\_ids = list(range(len(self.utility)))

while candidate\_ids:

idx = candidate\_ids[int(self.\_rng.random() \* len(candidate\_ids))]

candidate\_ids.remove(idx)

if bernoulli\_neg\_exp(-self.\_probabilities[idx], self.\_rng):

return self.candidates[idx] if self.candidates else idx

raise RuntimeError(f"No value to return. Probabilities: {self.\_probabilities}.")

class ExponentialCategorical(DPMechanism):

"""

The exponential mechanism for achieving differential privacy on categorical inputs, as first proposed by McSherry

and Talwar.

The exponential mechanism achieves differential privacy by randomly choosing an output value for a given input

value, with greater probability given to values 'closer' to the input, as measured by a given utility function.

Paper link: <https://www.cs.drexel.edu/~greenie/privacy/mdviadp.pdf>

Parameters

-----

`epsilon` : float

Privacy parameter :  $\epsilon$  for the mechanism. Must be in  $(0, \infty]$ .

`utility_list` : list of tuples

The utility list of the mechanism. Must be specified as a list of tuples, of the form `("value1", "value2",`

`utility)`, where each `value` is a string and `utility` is a strictly positive float. A `utility` must be

specified for every pair of values given in the `utility_list`.

`random_state` : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

"""

```

def __init__(self, *, epsilon, utility_list, random_state=None):
 super().__init__(epsilon=epsilon, delta=0.0, random_state=random_state)

 self._balanced_tree = False

 self._utility_values, self._sensitivity, self._domain_values = self._build_utility(utility_list)
 self._check_utility_full(self._domain_values)

 self._normalising_constant = self._build_normalising_constant()

def _build_utility(self, utility_list):
 if not isinstance(utility_list, list):
 raise TypeError("Utility must be given in a list")

 self._normalising_constant = None

 utility_values = {}
 domain_values = []
 sensitivity = 0

 for _utility_sub_list in utility_list:
 value1, value2, utility_value = _utility_sub_list

 if not isinstance(value1, str) or not isinstance(value2, str):
 raise TypeError("Utility keys must be strings")

 if not isinstance(utility_value, Real):
 raise TypeError("Utility value must be a number")

 if utility_value < 0.0:

```



```
raise ValueError("Utility values must be non-negative")
```

```
sensitivity = max(sensitivity, utility_value)
```

```
if value1 not in domain_values:
```

```
 domain_values.append(value1)
```

```
if value2 not in domain_values:
```

```
 domain_values.append(value2)
```

```
if value1 == value2:
```

```
 continue
```

```
if value1 < value2:
```

```
 utility_values[(value1, value2)] = utility_value
```

```
else:
```

```
 utility_values[(value2, value1)] = utility_value
```

```
self._utility_values = utility_values
```

```
self._sensitivity = sensitivity
```

```
self._domain_values = domain_values
```

```
return utility_values, sensitivity, domain_values
```

```
def _check_utility_full(self, domain_values):
```

```
 missing = []
```

```
 for val1 in domain_values:
```

```
 for val2 in domain_values:
```

```
if val1 >= val2:
```

```
 continue
```

```
if (val1, val2) not in self._utility_values:
```

```
 missing.append((val1, val2))
```

```
if missing:
```

```
 raise ValueError(f"Utility values missing: {missing}")
```

```
return True
```

```
@property
```

```
def utility_list(self):
```

```
 """Gets the utility list of the mechanism, in the same form as accepted by
 `.set_utility_list`.
```

```
 Returns
```

```

```

```
 utility_list : list of tuples (str, str, float), or None
```

```
 Returns a list of tuples of the form ("value1", "value2", utility), or `None` if the utility
 has not yet
```

```
 been set.
```

```
 """
```

```
 utility_list = []
```

```
 for _key, _utility in self._utility_values.items():
```

```
value1, value2 = _key
```

```
utility_list.append((value1, value2, _utility))
```

```
return utility_list
```

```
def _build_normalising_constant(self, re_eval=False):
```

```
 balanced_tree = True
```

```
 first_constant_value = None
```

```
 normalising_constant = {}
```

```
 for _base_leaf in self._domain_values:
```

```
 constant_value = 0.0
```

```
 for _target_leaf in self._domain_values:
```

```
 constant_value += self._get_prob(_base_leaf, _target_leaf)
```

```
 normalising_constant[_base_leaf] = constant_value
```

```
 if first_constant_value is None:
```

```
 first_constant_value = constant_value
```

```
 elif not np.isclose(constant_value, first_constant_value):
```

```
 balanced_tree = False
```

```
If the tree is balanced, we can eliminate the doubling factor
```

```
if balanced_tree and not re_eval:
```

```
 self._balanced_tree = True
```

```
 return self._build_normalising_constant(True)
```

```
 return normalising_constant
```

```
def _get_utility(self, value1, value2):
```

```
 if value1 == value2:
```

```
 return 0
```

```
 if value1 > value2:
```

```
 return self._get_utility(value1=value2, value2=value1)
```

```
 return self._utility_values[(value1, value2)]
```

```
def _get_prob(self, value1, value2):
```

```
 if value1 == value2:
```

```
 return 1.0
```

```
 balancing_factor = 1 if self._balanced_tree else 2
```

```
 return np.exp(- self.epsilon * self._get_utility(value1, value2) / balancing_factor /
self._sensitivity)
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 if not isinstance(value, str):
```

```
 raise TypeError("Value to be randomised must be a string")
```

```
if value not in self._domain_values:
 raise ValueError(f"Value \"{value}\" not in domain")
```

```
return True
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if not delta == 0:
```

```
 raise ValueError("Delta must be zero")
```

```
 return super()._check_epsilon_delta(epsilon, delta)
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(DPMechanism.variance)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(Binary.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
 unif_rv = self._rng.random() * self._normalising_constant[value]
```

```

cum_prob = 0
_target_value = None

for _target_value in self._normalising_constant.keys():
 cum_prob += self._get_prob(value, _target_value)

 if unif_rv <= cum_prob:
 return _target_value

return _target_value

```

```
class ExponentialHierarchical(ExponentialCategorical):
```

```
 r"""
```

Adaptation of the exponential mechanism to hierarchical data. Simplifies the process of specifying utility values,

as the values can be inferred from the hierarchy.

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

hierarchy : nested list of str

The hierarchy as specified as a nested list of string. Each string must be a leaf node, and each leaf node

must lie at the same depth in the hierarchy.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

## Examples

-----

Example hierarchies:

```
>>> flat_hierarchy = ["A", "B", "C", "D", "E"]
```

```
>>> nested_hierarchy = [["A"], ["B"], ["C"], ["D", "E"]]
```

```
"""
```

```
def __init__(self, *, epsilon, hierarchy, random_state=None):
```

```
 self.hierarchy = hierarchy
```

```
 utility_list = self._build_utility_list(self._build_hierarchy(hierarchy))
```

```
 super().__init__(epsilon=epsilon, utility_list=utility_list, random_state=random_state)
```

```
 self._list_hierarchy = None
```

```
def _build_hierarchy(self, nested_list, parent_node=None):
```

```
 if not isinstance(nested_list, list):
```

```
 raise TypeError("Hierarchy must be a list")
```

```
 if parent_node is None:
```

```
 parent_node = []
```

```
hierarchy = {}
```

```
for _i, _value in enumerate(nested_list):
```

```
 if isinstance(_value, str):
```

```
 hierarchy[_value] = parent_node + [_i]
```

```
 elif not isinstance(_value, list):
```

```
 raise TypeError("All leaves of the hierarchy must be a string " +
 "(see node " + str(parent_node + [_i]) + ")")
```

```
 else:
```

```
 hierarchy.update(self._build_hierarchy(_value, parent_node + [_i]))
```

```
self._check_hierarchy_height(hierarchy)
```

```
return hierarchy
```

```
@staticmethod
```

```
def _check_hierarchy_height(hierarchy):
```

```
 hierarchy_height = None
```

```
 for _value, _hierarchy_locator in hierarchy.items():
```

```
 if hierarchy_height is None:
```

```
 hierarchy_height = len(_hierarchy_locator)
```

```
 elif len(_hierarchy_locator) != hierarchy_height:
```

```
 raise ValueError(
```

```
 f"Leaves of the hierarchy must all be at the same level (node
{str(_hierarchy_locator)} is at "
```



```
 f"level {len(_hierarchy_locator)} instead of hierarchy height {hierarchy_height}"
)
```

```
@staticmethod
```

```
def _build_utility_list(hierarchy):
```

```
 if not isinstance(hierarchy, dict):
```

```
 raise TypeError("Hierarchy for _build_utility_list must be a dict")
```

```
 utility_list = []
```

```
 hierarchy_height = None
```

```
 for _root_value, _root_hierarchy_locator in hierarchy.items():
```

```
 if hierarchy_height is None:
```

```
 hierarchy_height = len(_root_hierarchy_locator)
```

```
 for _target_value, _target_hierarchy_locator in hierarchy.items():
```

```
 if _root_value >= _target_value:
```

```
 continue
```

```
 i = 0
```

```
 while (i < len(_root_hierarchy_locator) and
```

```
 _root_hierarchy_locator[i] == _target_hierarchy_locator[i]):
```

```
 i += 1
```

```
 utility_list.append([_root_value, _target_value, hierarchy_height - i])
```

```
return utility_list
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(DPMechanism.variance)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
MIT License
```

```
#
```

```
Copyright (C) IBM Corporation 2019
```

```
#
```

```
Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated
```

```
documentation files (the "Software"), to deal in the Software without restriction, including
without limitation the
```

```
rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
the Software, and to permit
```

```
persons to whom the Software is furnished to do so, subject to the following conditions:
```

```
#
```

```
The above copyright notice and this permission notice shall be included in all copies or
substantial portions of the
```

```
Software.
```

```
#
```

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

# SOFTWARE.

"""

The classic Gaussian mechanism in differential privacy, and its derivatives.

"""

from math import erf

from numbers import Real, Integral

import numpy as np

from diffprivlib.mechanisms.base import DPMechanism, bernoulli\_neg\_exp

from diffprivlib.mechanisms.geometric import Geometric

from diffprivlib.mechanisms.laplace import Laplace

from diffprivlib.utils import copy\_docstring

class Gaussian(DPMechanism):

r"""The Gaussian mechanism in differential privacy.

First proposed by Dwork and Roth in "The algorithmic foundations of differential privacy" [DR14]\_. Samples from the

Gaussian distribution are generated using two samples from `random.normalvariate` as detailed in [HB21b]\_, to

prevent against reconstruction attacks due to limited floating point precision.

## Parameters

-----

`epsilon` : float

Privacy parameter :  $\epsilon$  for the mechanism. Must be in  $(0, 1]$ . For  $\epsilon > 1$ , use

`.GaussianAnalytic`.

`delta` : float

Privacy parameter :  $\delta$  for the mechanism. Must be in  $(0, 1]$ .

`sensitivity` : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

`random_state` : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

## References

-----

.. [DR14] Dwork, Cynthia, and Aaron Roth. "The algorithmic foundations of differential privacy." Found. Trends

Theor. Comput. Sci. 9, no. 3-4 (2014): 211-407.

.. [HB21b] Holohan, Naoise, and Stefano Braghin. "Secure Random Sampling in Differential Privacy." arXiv preprint

arXiv:2107.10138 (2021).

"""

```
def __init__(self, *, epsilon, delta, sensitivity, random_state=None):
```

```
 super().__init__(epsilon=epsilon, delta=delta, random_state=random_state)
```

```
 self.sensitivity = self._check_sensitivity(sensitivity)
```

```
 self._scale = np.sqrt(2 * np.log(1.25 / self.delta)) * self.sensitivity / self.epsilon
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if epsilon == 0 or delta == 0:
```

```
 raise ValueError("Neither Epsilon nor Delta can be zero")
```

```
 if isinstance(epsilon, Real) and epsilon > 1.0:
```

```
 raise ValueError("Epsilon cannot be greater than 1. If required, use GaussianAnalytic instead.")
```

```
 return super()._check_epsilon_delta(epsilon, delta)
```

```
@classmethod
```

```
def _check_sensitivity(cls, sensitivity):
```

```
if not isinstance(sensitivity, Real):
 raise TypeError("Sensitivity must be numeric")
```

```
if sensitivity < 0:
 raise ValueError("Sensitivity must be non-negative")
```

```
return float(sensitivity)
```

```
def _check_all(self, value):
 super()._check_all(value)
 self._check_sensitivity(self.sensitivity)
```

```
if not isinstance(value, Real):
 raise TypeError("Value to be randomised must be a number")
```

```
return True
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
 return 0.0
```

```
@copy_docstring(Laplace.variance)
```

```
def variance(self, value):
 self._check_all(0)
```

```
return self._scale ** 2
```

```

@copy_docstring(Laplace.randomise)
def randomise(self, value):
 self._check_all(value)

 try:
 standard_normal = (self._rng.normalvariate(0, 1) + self._rng.normalvariate(0, 1)) /
np.sqrt(2)
 except AttributeError: # random_state is a np.random.RandomState
 standard_normal = (self._rng.standard_normal() + self._rng.standard_normal()) /
np.sqrt(2)

 return value + standard_normal * self._scale

```

```

class GaussianAnalytic(Gaussian):
 r"""The analytic Gaussian mechanism in differential privacy.

```

As first proposed by Balle and Wang in "Improving the Gaussian Mechanism for Differential Privacy: Analytical

Calibration and Optimal Denoising".

Paper link: <https://arxiv.org/pdf/1805.06530.pdf>

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

delta : float

Privacy parameter :math:`\delta` for the mechanism. Must be in  $(0, 1]$ .

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.`

"""

```
def __init__(self, *, epsilon, delta, sensitivity, random_state=None):
```

```
 super().__init__(epsilon=epsilon, delta=delta, sensitivity=sensitivity,
random_state=random_state)
```

```
 self._scale = self._find_scale()
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if epsilon == 0 or delta == 0:
```

```
 raise ValueError("Neither Epsilon nor Delta can be zero")
```

```
 return DPMechanism._check_epsilon_delta(epsilon, delta) # pylint:
disable=protected-access
```



```

def _check_all(self, value):
 super()._check_all(value)

 return True

def _find_scale(self):
 if self.sensitivity / self.epsilon == 0:
 return 0.0

 epsilon = self.epsilon
 delta = self.delta

 def phi(val):
 return (1 + erf(val / np.sqrt(2))) / 2

 def b_plus(val):
 return phi(np.sqrt(epsilon * val)) - np.exp(epsilon) * phi(- np.sqrt(epsilon * (val + 2))) -
delta

 def b_minus(val):
 return phi(- np.sqrt(epsilon * val)) - np.exp(epsilon) * phi(- np.sqrt(epsilon * (val + 2))) -
delta

 delta_0 = b_plus(0)

 if delta_0 < 0:
 target_func = b_plus

```

else:

target\_func = b\_minus

# Find the starting interval by doubling the initial size until the target\_func sign changes,  
as suggested

# in the paper

left = 0

right = 1

while target\_func(left) \* target\_func(right) > 0:

left = right

right \*= 2

# Binary search code copied from mechanisms.LaplaceBoundedDomain

old\_interval\_size = (right - left) \* 2

while old\_interval\_size > right - left:

old\_interval\_size = right - left

middle = (right + left) / 2

if target\_func(middle) \* target\_func(left) <= 0:

right = middle

if target\_func(middle) \* target\_func(right) <= 0:

left = middle

alpha = np.sqrt(1 + (left + right) / 4) + (-1 if delta\_0 < 0 else 1) \* np.sqrt((left + right) / 4)

```
return alpha * self.sensitivity / np.sqrt(2 * self.epsilon)
```

```
class GaussianDiscrete(DPMechanism):
```

```
 r"""The Discrete Gaussian mechanism in differential privacy.
```

As proposed by Canonne, Kamath and Steinke, re-purposed for approximate  
:math:`(\epsilon, \delta)`-differential  
privacy.

Paper link: <https://arxiv.org/pdf/2004.00010.pdf>

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

delta : float

Privacy parameter :math:`\delta` for the mechanism. Must be in  $(0, 1]$ .

sensitivity : int, default: 1

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

` ` random\_state` ` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, delta, sensitivity=1, random_state=None):
 super().__init__(epsilon=epsilon, delta=delta, random_state=random_state)
 self.sensitivity = self._check_sensitivity(sensitivity)
 self._scale = self._find_scale()
```

@classmethod

```
def _check_epsilon_delta(cls, epsilon, delta):
 if epsilon == 0 or delta == 0:
 raise ValueError("Neither Epsilon nor Delta can be zero")

 return super()._check_epsilon_delta(epsilon, delta)
```

@classmethod

```
def _check_sensitivity(cls, sensitivity):
 if not isinstance(sensitivity, Integral):
 raise TypeError("Sensitivity must be an integer")

 if sensitivity < 0:
 raise ValueError("Sensitivity must be non-negative")

 return sensitivity
```

```
def _check_all(self, value):
```

```
super()._check_all(value)

self._check_sensitivity(self.sensitivity)

if not isinstance(value, Integral):

 raise TypeError("Value to be randomised must be an integer")
```

```
return True
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
```

```
 return 0.0
```

```
@copy_docstring(Laplace.variance)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(Geometric.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
 if self._scale == 0:
```

```
 return value
```

```
 tau = 1 / (1 + np.floor(self._scale))
```

```
 sigma2 = self._scale ** 2
```

```

while True:

 geom_x = 0

 while bernoulli_neg_exp(tau, self._rng):

 geom_x += 1

 bern_b = self._rng.random() < 0.5

 if bern_b and not geom_x:

 continue

 lap_y = int((1 - 2 * bern_b) * geom_x)

 bern_c = bernoulli_neg_exp((abs(lap_y) - tau * sigma2) ** 2 / 2 / sigma2, self._rng)

 if bern_c:

 return value + lap_y

```

```

def _find_scale(self):

 """Determine the scale of the mechanism's distribution given epsilon and delta.

 """

 if self.sensitivity / self.epsilon == 0:

 return 0

```

```

def objective(sigma, epsilon_, delta_, sensitivity_):

 """Function for which we are seeking its root. """

 idx_0 = int(np.floor(epsilon_ * sigma ** 2 / sensitivity_ - sensitivity_ / 2))

 idx_1 = int(np.floor(epsilon_ * sigma ** 2 / sensitivity_ + sensitivity_ / 2))

 idx = 1

```

```

lhs, rhs, denom = float(idx_0 < 0), 0, 1
_term, diff = 1, 1

while _term > 0 and diff > 0:
 _term = np.exp(-idx ** 2 / 2 / sigma ** 2)

 if idx > idx_0:
 lhs += _term

 if idx_0 < -idx:
 lhs += _term

 if idx > idx_1:
 diff = -rhs
 rhs += _term
 diff += rhs

 denom += 2 * _term

 idx += 1

 if idx > 1e6:
 raise ValueError("Infinite sum not converging, aborting. Try changing the epsilon
and/or delta.")

return (lhs - np.exp(epsilon_) * rhs) / denom - delta_

epsilon = self.epsilon

```

```

delta = self.delta

sensitivity = self.sensitivity

Begin by locating the root within an interval $[2^{**i}, 2^{**i+1}]$
guess_0 = 1
f_0 = objective(guess_0, epsilon, delta, sensitivity)
pwr = 1 if f_0 > 0 else -1
guess_1 = 2 ** pwr
f_1 = objective(guess_1, epsilon, delta, sensitivity)

while f_0 * f_1 > 0:
 guess_0 *= 2 ** pwr
 guess_1 *= 2 ** pwr

 f_0 = f_1
 f_1 = objective(guess_1, epsilon, delta, sensitivity)

Find the root (sigma) using the bisection method
while not np.isclose(guess_0, guess_1, atol=1e-12, rtol=1e-6):
 guess_mid = (guess_0 + guess_1) / 2
 f_mid = objective(guess_mid, epsilon, delta, sensitivity)

 if f_mid * f_0 <= 0:
 f_1 = f_mid
 guess_1 = guess_mid

 if f_mid * f_1 <= 0:

```



```
f_0 = f_mid
```

```
guess_0 = guess_mid
```

```
return (guess_0 + guess_1) / 2
```

```
MIT License
```

```
#
```

```
Copyright (C) IBM Corporation 2019
```

```
#
```

```
Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated
```

```
documentation files (the "Software"), to deal in the Software without restriction, including
without limitation the
```

```
rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
the Software, and to permit
```

```
persons to whom the Software is furnished to do so, subject to the following conditions:
```

```
#
```

```
The above copyright notice and this permission notice shall be included in all copies or
substantial portions of the
```

```
Software.
```

```
#
```

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE
```

```
WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
```

```
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
```

```
SOFTWARE.
```

```
"""
```

```
The classic geometric mechanism for differential privacy, and its derivatives.
```

```
"""
```

```
from numbers import Integral
```

```
import numpy as np
```

```
from diffprivlib.mechanisms.base import DPMechanism, TruncationAndFoldingMixin
```

```
from diffprivlib.utils import copy_docstring
```

```
class Geometric(DPMechanism):
```

```
 """
```

```
 The classic geometric mechanism for differential privacy, as first proposed by Ghosh,
 Roughgarden and Sundararajan.
```

```
 Extended to allow for non-unity sensitivity.
```

```
 Paper link: https://arxiv.org/pdf/0811.2841.pdf
```

```
 Parameters
```

```

```

```
 epsilon : float
```

```
 Privacy parameter :math:`\epsilon` for the mechanism. Must be in $(0, \infty]$.
```

sensitivity : float, default: 1

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity=1, random_state=None):
```

```
 super().__init__(epsilon=epsilon, delta=0.0, random_state=random_state)
```

```
 self.sensitivity = self._check_sensitivity(sensitivity)
```

```
 self._scale = - self.epsilon / self.sensitivity if self.sensitivity > 0 else - float("inf")
```

```
@classmethod
```

```
def _check_sensitivity(cls, sensitivity):
```

```
 if not isinstance(sensitivity, Integral):
```

```
 raise TypeError("Sensitivity must be an integer")
```

```
 if sensitivity < 0:
```

```
 raise ValueError("Sensitivity must be non-negative")
```

```
 return sensitivity
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
self._check_sensitivity(self.sensitivity)
```

```
if not isinstance(value, Integral):
```

```
 raise TypeError("Value to be randomised must be an integer")
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if not delta == 0:
```

```
 raise ValueError("Delta must be zero")
```

```
 return super()._check_epsilon_delta(epsilon, delta)
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 return 0.0
```

```
@copy_docstring(DPMechanism.variance)
```

```
def variance(self, value):
```

```
 self._check_all(value)
```

```
 leading_factor = (1 - np.exp(self._scale)) / (1 + np.exp(self._scale))
```

```
 geom_series = np.exp(self._scale) / (1 - np.exp(self._scale))
```

```
 return 2 * leading_factor * (geom_series + 3 * (geom_series ** 2) + 2 * (geom_series **
3))
```

```
def randomise(self, value):
```

```
 """Randomise `value` with the mechanism.
```

```
 Parameters
```

```

```

```
 value : int
```

```
 The value to be randomised.
```

```
 Returns
```

```

```

```
 int
```

```
 The randomised value.
```

```
 """
```

```
 self._check_all(value)
```

```
 # Need to account for overlap of 0-value between distributions of different sign
```

```
 unif_rv = self._rng.random() - 0.5
```

```
 unif_rv *= 1 + np.exp(self._scale)
```

```
 sgn = -1 if unif_rv < 0 else 1
```

```
 # Use formula for geometric distribution, with ratio of $\exp(-\epsilon/\text{sensitivity})$
```

```
 return int(np.round(value + sgn * np.floor(np.log(sgn * unif_rv) / self._scale)))
```

```
class GeometricTruncated(Geometric, TruncationAndFoldingMixin):
```

```
"""
```

The truncated geometric mechanism, where values that fall outside a pre-described range are mapped back to the closest point within the range.

Parameters

```

```

`epsilon : float`

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

`sensitivity : float, default: 1`

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

`lower : int`

The lower bound of the mechanism.

`upper : int`

The upper bound of the mechanism.

`random_state : int or RandomState, optional`

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

```
"""
```

```
def __init__(self, *, epsilon, sensitivity=1, lower, upper, random_state=None):
```

```
 super().__init__(epsilon=epsilon, sensitivity=sensitivity, random_state=random_state)
```

```
TruncationAndFoldingMixin.__init__(self, lower=lower, upper=upper)
```

```
@classmethod
```

```
def _check_bounds(cls, lower, upper):
```

```
 if not isinstance(lower, Integral) and abs(lower) != float("inf"):
```

```
 raise TypeError(f"Lower bound must be integer-valued, got {lower}")
```

```
 if not isinstance(upper, Integral) and abs(upper) != float("inf"):
```

```
 raise TypeError(f"Upper bound must be integer-valued, got {upper}")
```

```
 return super()._check_bounds(lower, upper)
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(DPMechanism.bias)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 TruncationAndFoldingMixin._check_all(self, value)
```

```
 return True
```

```
@copy_docstring(Geometric.randomise)
```

```
def randomise(self, value):
 self._check_all(value)

 noisy_value = super().randomise(value)
 return int(np.round(self._truncate(noisy_value)))
```

```
class GeometricFolded(Geometric, TruncationAndFoldingMixin):
```

```
 r"""
```

The folded geometric mechanism, where values outside a pre-described range are folded back toward the domain around

the closest point within the domain.

Half-integer bounds are permitted.

Parameters

```

```

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

sensitivity : float, default: 1

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

lower : int or float

The lower bound of the mechanism. Must be integer or half-integer -valued.

upper : int or float



The upper bound of the mechanism. Must be integer or half-integer -valued.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity=1, lower, upper, random_state=None):
```

```
 super().__init__(epsilon=epsilon, sensitivity=sensitivity, random_state=random_state)
```

```
 TruncationAndFoldingMixin.__init__(self, lower=lower, upper=upper)
```

```
@classmethod
```

```
def _check_bounds(cls, lower, upper):
```

```
 if not np.isclose(2 * lower, np.round(2 * lower)) or not np.isclose(2 * upper, np.round(2 * upper)):
```

```
 raise ValueError("Bounds must be integer or half-integer floats")
```

```
 return super()._check_bounds(lower, upper)
```

```
def _fold(self, value):
```

```
 return super()._fold(int(np.round(value)))
```

```
@copy_docstring(DPMechanism.bias)
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```
@copy_docstring(DPMechanism.bias)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 TruncationAndFoldingMixin._check_all(self, value)
```

```
 return True
```

```
@copy_docstring(Geometric.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
 noisy_value = super().randomise(value)
```

```
 return int(np.round(self._fold(noisy_value)))
```

```
MIT License
```

```
#
```

```
Copyright (C) IBM Corporation 2019
```

```
#
```

```
Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated
```

```
documentation files (the "Software"), to deal in the Software without restriction, including
without limitation the
```

```
rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
the Software, and to permit
```

# persons to whom the Software is furnished to do so, subject to the following conditions:

#

# The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

# Software.

#

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

# SOFTWARE.

"""

The classic Laplace mechanism in differential privacy, and its derivatives.

"""

from numbers import Real

import numpy as np

from diffprivlib.mechanisms.base import DPMechanism, TruncationAndFoldingMixin

from diffprivlib.utils import copy\_docstring

class Laplace(DPMechanism):

"""

The classical Laplace mechanism in differential privacy.

First proposed by Dwork, McSherry, Nissim and Smith [DMNS16]\_, with support for (relaxed)

:math:`(\epsilon, \delta)`-differential privacy [HLM15]\_.

Samples from the Laplace distribution are generated using 4 uniform variates, as detailed in [HB21]\_, to prevent

against reconstruction attacks due to limited floating point precision.

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $[0, \infty]$ .

delta : float, default: 0.0

Privacy parameter :math:`\delta` for the mechanism. Must be in  $[0, 1]$ . Cannot be simultaneously zero with

`epsilon`.`

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

## References

-----

- .. [DMNS16] Dwork, Cynthia, Frank McSherry, Kobbi Nissim, and Adam Smith. "Calibrating noise to sensitivity in private data analysis." *Journal of Privacy and Confidentiality* 7, no. 3 (2016): 17-51.
- .. [HLM15] Holohan, Naoise, Douglas J. Leith, and Oliver Mason. "Differential privacy in metric spaces: Numerical, categorical and functional data under the one roof." *Information Sciences* 305 (2015): 256-268.
- .. [HB21] Holohan, Naoise, and Stefano Braghin. "Secure Random Sampling in Differential Privacy." *arXiv preprint arXiv:2107.10138* (2021).
- """
- ```
def __init__(self, *, epsilon, delta=0.0, sensitivity, random_state=None):
    super().__init__(epsilon=epsilon, delta=delta, random_state=random_state)
    self.sensitivity = self._check_sensitivity(sensitivity)
    self._scale = None
```
- @classmethod
- ```
def _check_sensitivity(cls, sensitivity):
 if not isinstance(sensitivity, Real):
 raise TypeError("Sensitivity must be numeric")
```

```

if sensitivity < 0:
 raise ValueError("Sensitivity must be non-negative")

return float(sensitivity)

def _check_all(self, value):
 super()._check_all(value)
 self._check_sensitivity(self.sensitivity)

if not isinstance(value, Real):
 raise TypeError("Value to be randomised must be a number")

return True

```

```

def bias(self, value):
 """Returns the bias of the mechanism at a given `value` .

```

Parameters

-----

value : int or float

The value at which the bias of the mechanism is sought.

Returns

-----

bias : float or None

The bias of the mechanism at `value` .

"""

return 0.0

def variance(self, value):

"""Returns the variance of the mechanism at a given `value` .

Parameters

-----

value : float

The value at which the variance of the mechanism is sought.

Returns

-----

bias : float

The variance of the mechanism at `value` .

"""

self.\_check\_all(0)

return 2 \* (self.sensitivity / (self.epsilon - np.log(1 - self.delta))) \*\* 2

@staticmethod

def \_laplace\_sampler(unif1, unif2, unif3, unif4):

return np.log(1 - unif1) \* np.cos(np.pi \* unif2) + np.log(1 - unif3) \* np.cos(np.pi \* unif4)

```
def randomise(self, value):
```

```
 """Randomise `value` with the mechanism.
```

```
 Parameters
```

```

```

```
 value : float
```

```
 The value to be randomised.
```

```
 Returns
```

```

```

```
 float
```

```
 The randomised value.
```

```
 """
```

```
 self._check_all(value)
```

```
 scale = self.sensitivity / (self.epsilon - np.log(1 - self.delta))
```

```
 standard_laplace = self._laplace_sampler(self._rng.random(), self._rng.random(),
self._rng.random(),
```

```
 self._rng.random())
```

```
 return value - scale * standard_laplace
```

```
class LaplaceTruncated(Laplace, TruncationAndFoldingMixin):
```



"""

The truncated Laplace mechanism, where values outside a pre-described domain are mapped to the closest point

within the domain.

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $[0, \infty]$ .

delta : float, default: 0.0

Privacy parameter :math:`\delta` for the mechanism. Must be in  $[0, 1]$ . Cannot be simultaneously zero with

$\epsilon$ .

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

lower : float

The lower bound of the mechanism.

upper : float

The upper bound of the mechanism.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

` ` random\_state` ` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, delta=0.0, sensitivity, lower, upper, random_state=None):
 super().__init__(epsilon=epsilon, delta=delta, sensitivity=sensitivity,
random_state=random_state)
```

```
 TruncationAndFoldingMixin.__init__(self, lower=lower, upper=upper)
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
```

```
 self._check_all(value)
```

```
 shape = self.sensitivity / self.epsilon
```

```
 return shape / 2 * (np.exp((self.lower - value) / shape) - np.exp((value - self.upper) /
shape))
```

```
@copy_docstring(Laplace.variance)
```

```
def variance(self, value):
```

```
 self._check_all(value)
```

```
 shape = self.sensitivity / self.epsilon
```

```
 variance = value ** 2 + shape * (self.lower * np.exp((self.lower - value) / shape)
- self.upper * np.exp((value - self.upper) / shape))
```

```
 variance += (shape ** 2) * (2 - np.exp((self.lower - value) / shape)
- np.exp((value - self.upper) / shape))
```

```
variance -= (self.bias(value) + value) ** 2
```

```
return variance
```

```
def _check_all(self, value):
```

```
 Laplace._check_all(self, value)
```

```
 TruncationAndFoldingMixin._check_all(self, value)
```

```
return True
```

```
@copy_docstring(Laplace.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
 noisy_value = super().randomise(value)
```

```
 return self._truncate(noisy_value)
```

```
class LaplaceFolded(Laplace, TruncationAndFoldingMixin):
```

```
 r"""
```

The folded Laplace mechanism, where values outside a pre-described domain are folded around the domain until they

fall within.

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $[0, \infty]$ .

delta : float, default: 0.0

Privacy parameter :math:`\delta` for the mechanism. Must be in  $[0, 1]$ . Cannot be simultaneously zero with

`epsilon`.`

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

lower : float

The lower bound of the mechanism.

upper : float

The upper bound of the mechanism.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.`

"""

```
def __init__(self, *, epsilon, delta=0.0, sensitivity, lower, upper, random_state=None):
 super().__init__(epsilon=epsilon, delta=delta, sensitivity=sensitivity,
random_state=random_state)
```

```
TruncationAndFoldingMixin.__init__(self, lower=lower, upper=upper)
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
```

```
 self._check_all(value)
```

```
 shape = self.sensitivity / self.epsilon
```

```
 bias = shape * (np.exp((self.lower + self.upper - 2 * value) / shape) - 1)
```

```
 bias /= np.exp((self.lower - value) / shape) + np.exp((self.upper - value) / shape)
```

```
 return bias
```

```
@copy_docstring(DPMechanism.variance)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 TruncationAndFoldingMixin._check_all(self, value)
```

```
 return True
```

```
@copy_docstring(Laplace.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
noisy_value = super().randomise(value)

return self._fold(noisy_value)
```

```
class LaplaceBoundedDomain(LaplaceTruncated):
```

```
 """
```

The bounded Laplace mechanism on a bounded domain. The mechanism draws values directly from the domain using

rejection sampling, without any post-processing [HABM20].

Parameters

```

```

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $[0, \infty]$ .

delta : float, default: 0.0

Privacy parameter :math:`\delta` for the mechanism. Must be in  $[0, 1]$ . Cannot be simultaneously zero with

`epsilon`.`

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty]$ .

lower : float

The lower bound of the mechanism.

upper : float

The upper bound of the mechanism.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

## References

-----

.. [HABM20] Holohan, Naoise, Spiros Antonatos, Stefano Braghin, and Pól Mac Aonghusa. "The Bounded Laplace Mechanism

in Differential Privacy." Journal of Privacy and Confidentiality 10, no. 1 (2020).

"""

```
def _find_scale(self):
```

```
 eps = self.epsilon
```

```
 delta = self.delta
```

```
 diam = self.upper - self.lower
```

```
 delta_q = self.sensitivity
```

```
def _delta_c(shape):
```

```
 if shape == 0:
```

```
 return 2.0
```

```
 return (2 - np.exp(- delta_q / shape) - np.exp(- (diam - delta_q) / shape)) / (1 - np.exp(-
diam / shape))
```

```
def _f(shape):
 return delta_q / (eps - np.log(_delta_c(shape)) - np.log(1 - delta))
```

```
left = delta_q / (eps - np.log(1 - delta))
```

```
right = _f(left)
```

```
old_interval_size = (right - left) * 2
```

```
while old_interval_size > right - left:
```

```
 old_interval_size = right - left
```

```
 middle = (right + left) / 2
```

```
 if _f(middle) >= middle:
```

```
 left = middle
```

```
 if _f(middle) <= middle:
```

```
 right = middle
```

```
return (right + left) / 2
```

```
def effective_epsilon(self):
```

```
 r"""Gets the effective epsilon of the mechanism, only for strict :math:`\epsilon` -
 differential privacy. Returns
```

```
 ``None`` if :math:`\delta` is non-zero.
```

```
 Returns
```

```

```

```
 float
```



The effective  $\epsilon$  parameter of the mechanism. Returns `None` if  $\delta$  is non-zero.

```
"""
```

```
if self._scale is None:
```

```
 self._scale = self._find_scale()
```

```
if self.delta > 0.0:
```

```
 return None
```

```
return self.sensitivity / self._scale
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
```

```
 self._check_all(value)
```

```
if self._scale is None:
```

```
 self._scale = self._find_scale()
```

```
bias = (self._scale - self.lower + value) / 2 * np.exp((self.lower - value) / self._scale) \
 - (self._scale + self.upper - value) / 2 * np.exp((value - self.upper) / self._scale)
```

```
bias /= 1 - np.exp((self.lower - value) / self._scale) / 2 \
 - np.exp((value - self.upper) / self._scale) / 2
```

```
return bias
```

```

@copy_docstring(Laplace.variance)
def variance(self, value):
 self._check_all(value)

 if self._scale is None:
 self._scale = self._find_scale()

 variance = value**2
 variance -= (np.exp((self.lower - value) / self._scale) * (self.lower ** 2)
 + np.exp((value - self.upper) / self._scale) * (self.upper ** 2)) / 2
 variance += self._scale * (self.lower * np.exp((self.lower - value) / self._scale)
 - self.upper * np.exp((value - self.upper) / self._scale))
 variance += (self._scale ** 2) * (2 - np.exp((self.lower - value) / self._scale)
 - np.exp((value - self.upper) / self._scale))
 variance /= 1 - (np.exp(-(value - self.lower) / self._scale)
 + np.exp(-(self.upper - value) / self._scale)) / 2

 variance -= (self.bias(value) + value) ** 2

 return variance

```

```

@copy_docstring(Laplace.randomise)
def randomise(self, value):
 self._check_all(value)

 if self._scale is None:

```

```

self._scale = self._find_scale()

value = max(min(value, self.upper), self.lower)
if np.isnan(value):
 return float("nan")

samples = 1

while True:
 try:
 unif = self._rng.random(4 * samples)
 except TypeError: # rng is secrets.SystemRandom
 unif = [self._rng.random() for _ in range(4 * samples)]
 noisy = value + self._scale * self._laplace_sampler(*np.array(unif).reshape(4, -1))

 if ((noisy >= self.lower) & (noisy <= self.upper)).any():
 idx = np.argmax((noisy >= self.lower) & (noisy <= self.upper))
 return noisy[idx]

 samples = min(100000, samples * 2)

```

```

class LaplaceBoundedNoise(Laplace):

```

```

 r"""

```

The Laplace mechanism with bounded noise, only applicable for approximate differential privacy ( $\delta > 0$ )

```

 [GDGK18]_.

```

Epsilon must be strictly positive,  $\epsilon > 0$ .  $\delta$  must be strictly in the interval (0, 0.5).

- For zero  $\epsilon$ , use :class:`.Uniform`.

- For zero  $\delta$ , use :class:`.Laplace`.

## Parameters

-----

**epsilon** : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

**delta** : float

Privacy parameter :math:`\delta` for the mechanism. Must be in  $(0, 0.5)$ .

**sensitivity** : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

**random\_state** : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.`

## References

-----

.. [GDGK18] Geng, Quan, Wei Ding, Ruiqi Guo, and Sanjiv Kumar. "Truncated Laplacian Mechanism for Approximate

Differential Privacy." arXiv preprint arXiv:1810.00877v1 (2018).

```

"""

def __init__(self, *, epsilon, delta, sensitivity, random_state=None):

 super().__init__(epsilon=epsilon, delta=delta, sensitivity=sensitivity,
random_state=random_state)

 self._noise_bound = None

@classmethod
def _check_epsilon_delta(cls, epsilon, delta):

 if epsilon == 0:

 raise ValueError("Epsilon must be strictly positive. For zero epsilon, use
:class:`.Uniform`.")

 if isinstance(delta, Real) and not 0 < delta < 0.5:

 raise ValueError("Delta must be strictly in the interval (0,0.5). For zero delta, use
:class:`.Laplace`.")

 return super()._check_epsilon_delta(epsilon, delta)

@copy_docstring(Laplace.bias)
def bias(self, value):

 return 0.0

@copy_docstring(DPMechanism.variance)
def variance(self, value):

 raise NotImplementedError

```

```

@copy_docstring(Laplace.randomise)

def randomise(self, value):
 self._check_all(value)

 if self._scale is None or self._noise_bound is None:
 self._scale = self.sensitivity / self.epsilon
 self._noise_bound = 0 if self._scale == 0 else \
 self._scale * np.log(1 + (np.exp(self.epsilon) - 1) / 2 / self.delta)

 if np.isnan(value):
 return float("nan")

 samples = 1

 while True:
 try:
 unif = self._rng.random(4 * samples)
 except TypeError: # rng is secrets.SystemRandom
 unif = [self._rng.random() for _ in range(4 * samples)]
 noisy = self._scale * self._laplace_sampler(*np.array(unif).reshape(4, -1))

 if ((noisy >= - self._noise_bound) & (noisy <= self._noise_bound)).any():
 idx = np.argmax((noisy >= - self._noise_bound) & (noisy <= self._noise_bound))
 return value + noisy[idx]

 samples = min(100000, samples * 2)

```

```
"""
```

The Snapping mechanism in differential privacy, which eliminates a weakness to floating point errors in the classic

Laplace mechanism with standard Laplace sampling.

```
"""
```

```
import struct
```

```
import numpy as np
```

```
try:
```

```
 from crlibm import log_rn # pylint: disable=no-name-in-module
```

```
except ModuleNotFoundError:
```

```
 log_rn = np.log
```

```
from diffprivlib.mechanisms import LaplaceTruncated
```

```
class Snapping(LaplaceTruncated):
```

```
 r"""
```

The Snapping mechanism for differential privacy.

First proposed by Ilya Mironov [Mir12]\_.

It eliminates a vulnerability stemming from the representation of reals as floating-point numbers in implementations

of the classic Laplace mechanism and its variants which use the inverse CDF of the Laplace distribution to sample

it. It causes a high degree of reduction in the granularity of the output.

For the most faithful implementation of the mechanism, the `crlibm` `` package should be installed.

## Parameters

-----

`epsilon` : float

Privacy parameter `math:\epsilon` for the mechanism. Must be in  $[\eta, \infty]$ , where `math:\eta` is the

machine epsilon of the floating point type.

`sensitivity` : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

`lower` : float

The lower bound of the mechanism.

`upper` : float

The upper bound of the mechanism.

`random_state` : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` `` has to be fixed to an integer.

## References



```

.. [Mir12] Mironov, Ilya. "On significance of the least significant bits for differential
privacy." Proceedings of
the 2012 ACM conference on Computer and communications security (2012).

"""

def __init__(self, *, epsilon, sensitivity, lower, upper, random_state=None):
 super().__init__(epsilon=epsilon, sensitivity=sensitivity, delta=0.0, lower=lower,
upper=upper,
 random_state=random_state)
 self._bound = self._scale_bound()

 @classmethod
 def _check_epsilon_delta(cls, epsilon, delta):
 epsilon, delta = super()._check_epsilon_delta(epsilon, delta)

 machine_epsilon = np.finfo(float).epsneg
 if epsilon <= 2 * machine_epsilon:
 raise ValueError("Epsilon must be at least as large as twice the machine epsilon for
the floating point "
 "type, as the effective epsilon must be non-negative")

 return epsilon, delta

 def _scale_bound(self):
 """

```

Scales the lower and upper bounds to be proportionate to sensitivity 1, and symmetrical about 0.

For sensitivity 0, only centres the bound, as scaling up and down is not defined.

Returns

-----

float

A symmetric bound around 0 scaled to sensitivity 1

"""

```
if self.sensitivity == 0:
```

```
 return (self.upper - self.lower) / 2.0
```

```
return (self.upper - self.lower) / 2.0 / self.sensitivity
```

```
def _truncate(self, value):
```

```
 if value > self._bound:
```

```
 return self._bound
```

```
 if value < -self._bound:
```

```
 return -self._bound
```

```
 return value
```

```
def bias(self, value):
```

```
 raise NotImplementedError
```

```
def variance(self, value):
```

```
raise NotImplementedError
```

```
def effective_epsilon(self):
```

```
 r"""
```

Returns the effective value used in the Snapping mechanism to give the required  
:math:`\epsilon` -DP, based on

the bounds and the machine epsilon.

Based on section 5.2 of [Mir12].

Returns

-----

float

The effective value of :math:`\epsilon`

```
 """
```

```
 machine_epsilon = np.finfo(float).epsneg
```

```
 return (self.epsilon - 2 * machine_epsilon) / (1 + 12 * self._bound * machine_epsilon)
```

```
def _scale_and_offset_value(self, value):
```

```
 """
```

Centre value around 0 with symmetric bound and scale to sensitivity 1

Parameters

-----

value : float

value to be scaled

Returns

-----

float

value offset to be centered on 0 and scaled to sensitivity 1

"""

value\_scaled = value / self.sensitivity

return value\_scaled - self.\_bound - (self.lower / self.sensitivity)

def \_reverse\_scale\_and\_offset\_value(self, value):

return (value + self.\_bound) \* self.sensitivity + self.lower

@staticmethod

def \_get\_nearest\_power\_of\_2(x):

def float\_to\_bits(d):

s = struct.pack('>d', d)

return struct.unpack('>q', s)[0]

def bits\_to\_float(b):

s = struct.pack('>q', b)

return struct.unpack('>d', s)[0]

bits = float\_to\_bits(x)

mantissa\_size = np.finfo(float).nmant

if bits % (1 << mantissa\_size) == 0:

return x

```
return bits_to_float(((bits >> mantissa_size) + 1) << mantissa_size)
```

```
def _round_to_nearest_power_of_2(self, value, lambda_):
```

```
 """ Performs the rounding step from [Mir12]_ with ties resolved towards $+\infty$
```

Parameters

-----

value : float

Value to be rounded

Returns

-----

float

Rounded value

"""

```
if self.epsilon == float('inf'): # infinitely small rounding
```

```
 return value
```

```
remainder = value % lambda_
```

```
if remainder > lambda_ / 2:
```

```
 return value - remainder + lambda_
```

```
if remainder == lambda_ / 2:
```

```
 return value + remainder
```

```
return value - remainder
```

```
def _uniform_sampler(self):
```

"""

Uniformly sample the full domain of floating-point numbers between (0, 1), rather than only multiples of  $2^{-53}$ .

A uniform distribution over  $D \cap (0, 1)$  can be generated by independently sampling an exponent

from the geometric distribution with parameter .5 and a significand by drawing a uniform string from

$\{0, 1\}^{52}$  [Mir12]\_

Based on code recipe in Python standard library documentation [Py21]\_.

Returns

-----

float

A value sampled from float in (0, 1) with probability proportional to the size of the infinite-precision

real interval each float represents

References

-----

.. [Py21] The Python Standard Library. "random — Generate pseudo-random numbers", 2021

<https://docs.python.org/3/library/random.html#recipes>

"""

`mantissa_size = np.finfo(float).nmant`

`mantissa = 1 << mantissa_size | self._getrandbits(mantissa_size)`

```

exponent = -(mantissa_size + 1)

x = 0

while not x:

 x = self._getrandbits(32)

 exponent += x.bit_length() - 32

return np.ldexp(mantissa, exponent)

```

```

def _getrandbits(self, bits):

 try:

 return self._rng.getrandbits(bits)

 except AttributeError:

 return self._rng.randint(0, 2 ** bits)

```

@staticmethod

```

def _laplace_sampler(unif_bit, unif):

```

```

 r"""

```

Laplace inverse CDF random sampling implementation which uses full domain uniform sampling and exact log

implementation from `crlibm` (if installed), as mentioned in [Mir12].

Outputs a random value scaled according to privacy budget and sensitivity 1, as bounds and input are scaled to

sensitivity 1 before Laplacian noise is added.

Returns

```

```

float

Random value from Laplace distribution scaled according to  $\epsilon$

```
"""
```

```
laplace = (-1) ** unif_bit * log_rn(unif)
```

```
return laplace
```

```
def randomise(self, value):
```

```
 """Randomise `value` with the mechanism.
```

```
 Parameters
```

```

```

```
 value : float
```

```
 The value to be randomised.
```

```
 Returns
```

```

```

```
 float
```

```
 The randomised value.
```

```
"""
```

```
self._check_all(value)
```

```
if self.sensitivity == 0:
```

```
 return self._truncate(value)
```

```
value_scaled_offset = self._scale_and_offset_value(value)
```

```
value_clamped = self._truncate(value_scaled_offset)
```



```
scale = 1.0 / self.effective_epsilon() # everything is already scaled to sensitivity 1
lambda_ = self._get_nearest_power_of_2(scale)
laplace = scale * self._laplace_sampler(self._getrandbits(1), self._uniform_sampler())
value_rounded = self._round_to_nearest_power_of_2(value_clamped + laplace,
lambda_)
return self._reverse_scale_and_offset_value(self._truncate(value_rounded))
```

# MIT License

#

# Copyright (C) IBM Corporation 2019

#

# Permission is hereby granted, free of charge, to any person obtaining a copy of this  
software and associated

# documentation files (the "Software"), to deal in the Software without restriction, including  
without limitation the

# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of  
the Software, and to permit

# persons to whom the Software is furnished to do so, subject to the following conditions:

#

# The above copyright notice and this permission notice shall be included in all copies or  
substantial portions of the

# Software.

#

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR  
IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND  
NONINFRINGEMENT. IN NO EVENT SHALL THE

```
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
```

```
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
```

```
SOFTWARE.
```

```
"""
```

```
The staircase mechanism in differential privacy.
```

```
"""
```

```
import secrets
```

```
from numbers import Real
```

```
import numpy as np
```

```
from diffprivlib.mechanisms.laplace import Laplace
```

```
from diffprivlib.utils import copy_docstring
```

```
class Staircase(Laplace):
```

```
 r"""
```

```
 The staircase mechanism in differential privacy.
```

```
 The staircase mechanism is an optimisation of the classical Laplace Mechanism
 (:class:`.Laplace`), described as a
```

```
 "geometric mixture of uniform random variables".
```

```
 Paper link: https://arxiv.org/pdf/1212.1186.pdf
```

```
 Parameters
```

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

sensitivity : float

The sensitivity of the mechanism. Must be in  $[0, \infty)$ .

gamma : float, default:  $1 / (1 + \exp(\epsilon/2))$

Value of the tuning parameter gamma for the mechanism. Must be in  $[0, 1]$ .

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

`random_state` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, sensitivity, gamma=None, random_state=None):
 super().__init__(epsilon=epsilon, delta=0, sensitivity=sensitivity,
random_state=random_state)
```

```
 self.gamma = self._check_gamma(gamma, epsilon=self.epsilon)
```

```
 if isinstance(self._rng, secrets.SystemRandom):
```

```
 self._rng = np.random.default_rng()
```

```
@classmethod
```

```
def _check_gamma(cls, gamma, epsilon=None):
```

```
 if gamma is None and epsilon is not None:
```

```
gamma = 1 / (1 + np.exp(epsilon / 2))
```

```
if not isinstance(gamma, Real):
```

```
 raise TypeError("Gamma must be numeric")
```

```
if not 0.0 <= gamma <= 1.0:
```

```
 raise ValueError("Gamma must be in [0,1]")
```

```
return float(gamma)
```

```
@copy_docstring(Laplace._check_all)
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 self._check_gamma(self.gamma)
```

```
return True
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
```

```
 if not delta == 0:
```

```
 raise ValueError("Delta must be zero")
```

```
return super()._check_epsilon_delta(epsilon, delta)
```

```
@copy_docstring(Laplace.bias)
```

```
def bias(self, value):
```

```
 return 0.0
```

```

@copy_docstring(Laplace.variance)
def variance(self, value):
 raise NotImplementedError

@copy_docstring(Laplace.randomise)
def randomise(self, value):
 self._check_all(value)

 sign = -1 if self._rng.random() < 0.5 else 1
 geometric_rv = self._rng.geometric(1 - np.exp(- self.epsilon)) - 1
 unif_rv = self._rng.random()
 binary_rv = 0 if self._rng.random() < self.gamma / (self.gamma +
 (1 - self.gamma) * np.exp(- self.epsilon)) else 1

 return value + sign * ((1 - binary_rv) * ((geometric_rv + self.gamma * unif_rv) *
self.sensitivity) +
 binary_rv * ((geometric_rv + self.gamma + (1 - self.gamma) * unif_rv) *
self.sensitivity))

MIT License
#
Copyright (C) IBM Corporation 2019
#
Permission is hereby granted, free of charge, to any person obtaining a copy of this
software and associated

```

# documentation files (the "Software"), to deal in the Software without restriction, including without limitation the

# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit

# persons to whom the Software is furnished to do so, subject to the following conditions:

#

# The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

# Software.

#

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

# SOFTWARE.

""

The uniform mechanism in differential privacy.

""

from numbers import Real

from diffprivlib.mechanisms.base import DPMechanism

from diffprivlib.mechanisms.laplace import Laplace

from diffprivlib.utils import copy\_docstring

```
class Uniform(DPMechanism):
```

```
 r"""
```

```
 The Uniform mechanism in differential privacy.
```

```
 This emerges as a special case of the :class:`.LaplaceBoundedNoise` mechanism when
 epsilon = 0.
```

```
 Paper link: https://arxiv.org/pdf/1810.00877.pdf
```

```
 Parameters
```

```

```

```
 delta : float
```

```
 Privacy parameter :math:`\delta` for the mechanism. Must be in (0, 0.5].
```

```
 sensitivity : float
```

```
 The sensitivity of the mechanism. Must be in $[0, \infty)$.
```

```
 random_state : int or RandomState, optional
```

```
 Controls the randomness of the mechanism. To obtain a deterministic behaviour
 during randomisation,
```

```
 ``random_state`` has to be fixed to an integer.
```

```
 """
```

```
 def __init__(self, *, delta, sensitivity, random_state=None):
```

```
 super().__init__(epsilon=0.0, delta=delta, random_state=random_state)
```

```
 self.sensitivity = self._check_sensitivity(sensitivity)
```

```
@classmethod
def _check_epsilon_delta(cls, epsilon, delta):
 if not epsilon == 0:
 raise ValueError("Epsilon must be strictly zero.")

 if not 0 < delta <= 0.5:
 raise ValueError("Delta must be in the half-open interval (0, 0.5]")

 return super()._check_epsilon_delta(epsilon, delta)
```

```
@classmethod
def _check_sensitivity(cls, sensitivity):
 if not isinstance(sensitivity, Real):
 raise TypeError("Sensitivity must be numeric")

 if sensitivity < 0:
 raise ValueError("Sensitivity must be non-negative")

 return float(sensitivity)
```

```
@copy_docstring(Laplace.bias)
def bias(self, value):
 return 0.0
```

```
@copy_docstring(Laplace.variance)
def variance(self, value):
```



```
self._check_all(value)
```

```
return (self.sensitivity / self.delta) ** 2 / 12
```

```
def _check_all(self, value):
```

```
 super()._check_all(value)
```

```
 self._check_sensitivity(self.sensitivity)
```

```
 if not isinstance(value, Real):
```

```
 raise TypeError("Value to be randomised must be a number")
```

```
 return True
```

```
@copy_docstring(Laplace.randomise)
```

```
def randomise(self, value):
```

```
 self._check_all(value)
```

```
 unif_rv = 2 * self._rng.random() - 1
```

```
 unif_rv *= self.sensitivity / self.delta / 2
```

```
 return value + unif_rv
```

```
MIT License
```

```
#
```

```
Copyright (C) IBM Corporation 2019
```

```
#
```

# Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated

# documentation files (the "Software"), to deal in the Software without restriction, including without limitation the

# rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit

# persons to whom the Software is furnished to do so, subject to the following conditions:

#

# The above copyright notice and this permission notice shall be included in all copies or substantial portions of the

# Software.

#

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

# WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,

# TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE

# SOFTWARE.

""

The vector mechanism in differential privacy, for producing perturbed objectives

""

from numbers import Real

import numpy as np

```
from diffprivlib.mechanisms.base import DPMechanism
from diffprivlib.utils import copy_docstring
```

```
class Vector(DPMechanism):
```

```
 r"""
```

The vector mechanism in differential privacy.

The vector mechanism is used when perturbing convex objective functions.

Full paper: <http://www.jmlr.org/papers/volume12/chaudhuri11a/chaudhuri11a.pdf>

Parameters

-----

epsilon : float

Privacy parameter :math:`\epsilon` for the mechanism. Must be in  $(0, \infty]$ .

function\_sensitivity : float

The function sensitivity of the mechanism. Must be in  $[0, \infty)$ .

data\_sensitivity : float, default: 1.0

The data sensitivity of the mechanism. Must be in  $[0, \infty)$ .

dimension : int

Function input dimension. This dimension relates to the size of the input vector of the function being

considered by the mechanism. This corresponds to the size of the random vector produced by the mechanism. Must

be in  $[1, \infty)$ .

alpha : float, default: 0.01

Regularisation parameter. Must be in  $(0, \infty)$ .

n : int, default: 1

Size of the training dataset, required to calibrate the influence of the random vector in the objective.

random\_state : int or RandomState, optional

Controls the randomness of the mechanism. To obtain a deterministic behaviour during randomisation,

``random\_state`` has to be fixed to an integer.

"""

```
def __init__(self, *, epsilon, function_sensitivity, data_sensitivity=1.0, dimension,
alpha=0.01, n=1,
```

```
 random_state=None):
```

```
 super().__init__(epsilon=epsilon, delta=0.0, random_state=random_state)
```

```
 self.function_sensitivity, self.data_sensitivity =
self._check_sensitivity(function_sensitivity,
 data_sensitivity)
```

```
 self.dimension = self._check_dimension(dimension)
```

```
 self.alpha = self._check_alpha(alpha)
```

```
 self.n = int(n)
```

```
@classmethod
```

```
def _check_epsilon_delta(cls, epsilon, delta):
 if not delta == 0:
 raise ValueError("Delta must be zero")

 return super()._check_epsilon_delta(epsilon, delta)
```

```
@classmethod
```

```
def _check_alpha(cls, alpha):
 if not isinstance(alpha, Real):
 raise TypeError("Alpha must be numeric")

 if alpha <= 0:
 raise ValueError("Alpha must be strictly positive")

 return alpha
```

```
@classmethod
```

```
def _check_dimension(cls, vector_dim):
 if not isinstance(vector_dim, Real) or not np.isclose(vector_dim, int(vector_dim)):
 raise TypeError("d must be integer-valued")
 if int(vector_dim) < 1:
 raise ValueError("d must be strictly positive")

 return int(vector_dim)
```

```
@classmethod
```

```

def _check_sensitivity(cls, function_sensitivity, data_sensitivity):
 if not isinstance(function_sensitivity, Real) or not isinstance(data_sensitivity, Real):
 raise TypeError("Sensitivities must be numeric")

 if function_sensitivity < 0 or data_sensitivity < 0:
 raise ValueError("Sensitivities must be non-negative")

 return function_sensitivity, data_sensitivity

```

```

def _check_all(self, value):
 super()._check_all(value)
 self._check_alpha(self.alpha)
 self._check_sensitivity(self.function_sensitivity, self.data_sensitivity)
 self._check_dimension(self.dimension)

 if self.n < 1:
 raise ValueError(f"n must be strictly positive, got {self.n}")

 if not callable(value):
 raise TypeError("Value to be randomised must be a function")

 return True

```

```

@copy_docstring(DPMechanism.bias)

```

```

def bias(self, value):
 raise NotImplementedError

```

```
@copy_docstring(DPMechanism.variance)
```

```
def variance(self, value):
```

```
 raise NotImplementedError
```

```
def randomise(self, value):
```

```
 """Randomise `value` with the mechanism.
```

If `value` is a method of two outputs, they are taken as `f` and `fprime` (i.e., its gradient), and both are

perturbed accordingly.

Parameters

-----

value : method

The function to be randomised.

Returns

-----

method

The randomised method.

"""

```
self._check_all(value)
```

```
epsilon_p = self.epsilon - 2 * np.log(1 + self.function_sensitivity * self.data_sensitivity /
self.alpha)
```

```

delta = 0

if epsilon_p <= 0:
 delta = (self.function_sensitivity * self.data_sensitivity / np.exp(1(self.epsilon / 4)
 - self.alpha) / self.n
 epsilon_p = self.epsilon / 2

scale = self.data_sensitivity * 2 / epsilon_p

try:
 normed_noisy_vector = self._rng.standard_normal((self.dimension, 4)).sum(axis=1) /
2
 noisy_norm = self._rng.gamma(self.dimension / 4, scale, 4).sum()
except AttributeError: # rng is secrets.SystemRandom
 normed_noisy_vector = np.reshape([self._rng.normalvariate(0, 1) for _ in
range(self.dimension * 4)],
 (-1, 4)).sum(axis=1) / 2
 noisy_norm = sum(self._rng.gammavariate(self.dimension / 4, scale) for _ in range(4))
if scale > 0 else 0.0

norm = np.linalg.norm(normed_noisy_vector, 2)
normed_noisy_vector = normed_noisy_vector / norm * noisy_norm

def output_func(*args):
 input_vec = args[0]

 func = value(*args)

```



```
if isinstance(func, tuple):
 func, grad = func
else:
 grad = None

func += np.dot(normed_noisy_vector, input_vec) / self.n
func += 0.5 * delta * np.dot(input_vec, input_vec)

if grad is not None:
 grad += normed_noisy_vector / self.n + delta * input_vec

 return func, grad

return func

return output_func
```